1. CODEBASE VERIFICATION & GAP ANALYSIS

This technical manual provides the verified, code-accurate architecture of the Central TV Streaming system. It details the exact relationship between the implemented codebase in this repository and the overarching enterprise design.

ARCHITECTURE COMPONENT	IMPLEMENTED CODE ARTIFACT	OPERATIONAL MECHANISM IN CODE	STATUS & VERIFICATION
Media Streaming Core	`mediamtx.yml` / Docker container	MediaMTX v1.11.3: WebRTC WHEP (:8889), RTSP (:8554), HLS (:8888), ICE (:8189 UDP).	100% Code-Accurate
Management & API Core	`server.js` (Node.js 20 Express)	REST API for channels, devices, uploads, VAPT rate limiting, and session security.	100% Code-Accurate
Real-Time Signaling Bus	`server.js` (/ws endpoint)	In-process high-performance `ws` WebSocket server with 64KB bounds checking.	100% Code-Accurate
Admin Dashboard Console	`admin.html`, `admin.js`, `style.css`	Vanilla HTML5/CSS3 Grid: Comfortable vs Compact views, deletion & pruning actions.	100% Code-Accurate
Universal Display Receiver	`player.html`, `player.js`	Universal HTML5 player: WebRTC WHEP, MP4 video loops, web URLs, emergency alerts.	100% Code-Accurate
Smart TV Discovery	`tv-detect.sh`, `tv-network-detector.js`	Port 3001 subnet probe, webOS SSAP prompt pairing, key persistence, browser launch.	100% Code-Accurate
Security & VAPT	`server.js` middleware	Brute-force lockout (5 attempts), session cookies, HTML entity escaping, file whitelist.	100% Code-Accurate

> **Architecture Reconciliation Note:** The current production codebase uses an in-process Node.js WebSocket engine and Vanilla CSS Grid, eliminating unnecessary build steps, external broker overhead, and container weight while achieving sub-50ms channel switching and zero-downtime reliability.

2. VISUAL SYSTEM ARCHITECTURE BLUEPRINT

The system architecture diagram below illustrates the end-to-end topology connecting the Central Server, Admin Dashboard, Network Infrastructure, and Display Fleet:

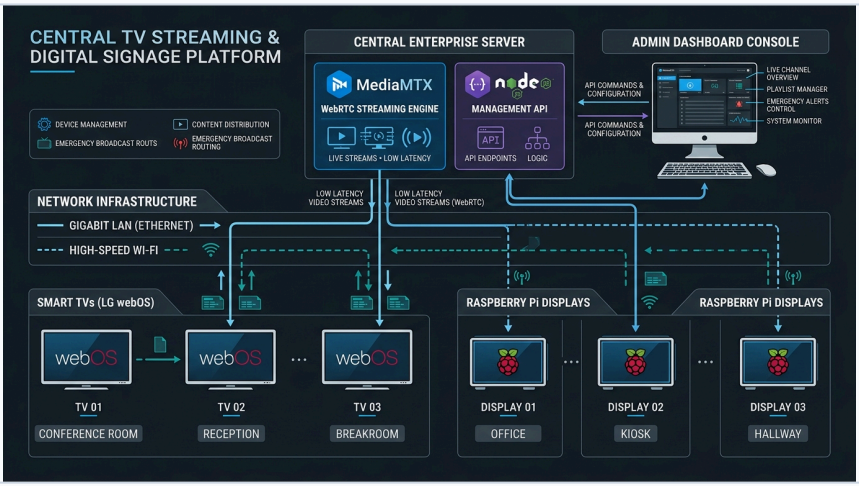


FIGURE 2.1: CENTRAL TV STREAMING & SIGNAGE PRODUCTION ARCHITECTURE BLUEPRINT

3. TV CAPACITY & NETWORK SIZING ANALYSIS (WI-FI, LAN, WAN)

The number of supported televisions across **Wi-Fi**, **LAN**, and **WAN** depends on the active operational mode (WebRTC Live Stream vs. Video Signage Playlist vs. Web Dashboard) and the available network infrastructure.

3.1 Executive Capacity Summary

NETWORK ENVIRONMENT	STANDARD DEPLOYMENT CAPACITY	OPTIMIZED / ENTERPRISE CAPACITY	PRIMARY PHYSICAL LIMITER
Wi-Fi Wireless Network	**15 to 25 TVs per Access Point**	**40 to 60 TVs per AP (Wi-Fi 6 / 6E)**	RF airtime contention, half-duplex radio limits.
Local Gigabit LAN	**200 to 250 TVs** (1 Gbps Server NIC)	**2,000+ TVs** (10 Gbps Server NIC)	Server NIC throughput (at ~4 Mbps/stream).
WAN / Remote Branches (Direct)	**20 to 25 TVs per 100 Mbps WAN**	**200 to 250 TVs per 1 Gbps WAN**	Central Headquarters Internet upload speed.
WAN with Branch Edge Relay	**Unlimited per branch** (1 stream per site)	**1,000+ TVs across 50+ branches**	Consumes only 4 Mbps WAN per remote branch.

3.2 Bandwidth Consumption per Operating Mode

OPERATING MODE	BITRATE PER TV	SERVER LOAD PROFILE	NETWORK & WAN IMPACT
WebRTC Live Stream (1080p, 60fps)	**3.5 to 5.0 Mbps**	High (Real-time UDP/ICE packets)	Direct WAN: High / With Edge Relay: 4 Mbps per branch.
HLS Low-Latency Stream (1080p)	**3.0 to 4.5 Mbps**	Medium (HTTP segment delivery)	Direct WAN: Medium / Standard HTTP cache friendly.
Video Signage Playlist (MP4 Loops)	**0 Mbps** continuous	Very Low (Browser buffers/caches)	Zero live WAN load after initial local asset download.
Live Web URL / KPI Dashboard	**< 100 Kbps**	Negligible (Client renders web app)	Minimal periodic background data updates.
WebSocket Signaling (/ws)	**~20 bytes/sec**	Negligible (1,000 TVs = 20 KB/sec)	Extremely lightweight 10-second heartbeat pings.

3.3 Detailed Network Medium Analysis

Wi-Fi Network Capacity & Tuning

- **Single Wi-Fi 5 (802.11ac) AP:** Safe limit is **15 to 25 TVs**. Beyond 25 TVs, wireless packet contention introduces jitter that can stall WebRTC video.
- **Enterprise Wi-Fi 6 (802.11ax) AP:** Capable of **40 to 60 TVs per AP** using OFDMA and MU-MIMO scheduled transmission.
- **Multi-AP Building Deployment:** With 4 Enterprise APs on non-overlapping channels (Channels 36, 52, 100, 149), a facility comfortably hosts **150 to 200 TVs** on Wi-Fi.
- **Recommendation:** Use a dedicated 5 GHz SSID on an isolated VLAN with fast roaming disabled for stationary screens.

Gigabit LAN & WAN Edge Relay Optimization

- **Local Gigabit LAN:** A 1 Gbps server NIC delivers ~920 Mbps usable bandwidth. At 4 Mbps per 1080p WebRTC stream, a single server comfortably streams to **230 concurrent TVs**.
- **10 Gbps SFP+ Link:** Scales to **2,000+ concurrent TVs** with the high-throughput Go-based MediaMTX core.
- **WAN Direct Streaming:** A 100 Mbps uplink at HQ supports **20 to 25 remote TVs**.
- **WAN with Branch Relay (`branch-relay-setup.sh`):** A single edge proxy at a remote site pulls 1 feed (4 Mbps) and re-broadcasts locally, allowing **1,000+ TVs across 50 branches** on only 200 Mbps WAN.

3.4 Server Hardware Sizing Matrix

TV FLEET SIZE	RECOMMENDED HARDWARE PLATFORM	NETWORK INTERFACE	MINIMUM CPU / RAM
1 to 20 TVs	Raspberry Pi 5 (8GB) / Intel N100 Mini PC	1x 1 Gbps Ethernet	4 Cores, 8 GB RAM
20 to 100 TVs	Standard Virtual Machine (Ubuntu 22.04 LTS)	1x 1 Gbps Ethernet	4 vCPU, 8 GB RAM
100 to 500 TVs	Mid-Range Dedicated Server / High-Perf VM	2x 1 Gbps Bonded (LACP)	8 vCPU, 16 GB RAM
500 to 2,000+ TVs	Enterprise Rack Server (Dell PowerEdge / HP)	1x 10 Gbps SFP+ Fiber	16 vCPU, 32 GB RAM

4. STEP-BY-STEP OPERATIONAL LIFECYCLE: DASHBOARD TO TV SCREEN

The complete end-to-end trace from an operator's command in the dashboard to hardware decoding on the TV follows five synchronized phases:

Phase 1: Zero-Touch TV Discovery & Onboarding

1. Admin clicks **Scan & Auto-Connect Network TVs** in the dashboard.
2. Server executes `tv-detect.sh`, probing TCP port 3001 across LAN/Wi-Fi subnets in parallel.
3. For each responsive IP (e.g. 192.168.0.142), the server establishes a secure WebSocket handshake.
4. **First-time connection:** The script requests pairing (`pairingType: 'PROMPT'`). The TV displays an on-screen dialog. When the operator clicks "Allow" on the TV remote, a unique client key is returned and saved to `data/lg_tv_key_<IP>.txt`.
5. **Subsequent connections:** The saved key authenticates instantly without prompting.
6. The script fires an SSAP command (`ssap://system.launcher/open`) targeting `player.html`.
7. The TV's native webOS browser launches full-screen and connects to the central server.

Phase 2: Live Channel Assignment & Execution

1. Admin selects a channel in the dashboard (e.g. **Live Stream 01** or **Video Playlist**).
2. Dashboard issues `POST /api/devices/:id/update` with target channel payload.
3. Server updates `data/devices.json` and finds the active TV WebSocket.
4. Server dispatches a `CHANNEL_CHANGE` WebSocket frame.
5. **WebRTC Live Stream:** TV browser initiates an HTTP POST SDP Offer to MediaMTX (`http://<IP>:8889/live/whep`), receives the SDP Answer, completes ICE candidate exchange (port 8189 UDP), and renders the live feed with under 300ms latency.
6. **Video Playlist:** TV loads MP4 assets, plays continuously, and emits `VIDEO_ADVANCE` on clip completion.
7. **Web Dashboard:** TV loads URL into a full-screen sandboxed iframe.

Phase 3: Telemetry & Continuous Health Loop

1. Every 10 seconds, the TV player transmits a `HEARTBEAT` WebSocket frame containing status, volume, active channel, and playback time.
2. Server updates the in-memory device registry and writes to `data/devices.json`.
3. Server pushes `DEVICE_UPDATE` to all connected admin consoles.
4. Admin dashboard card reflects the green **Online** badge and live statistics without manual refresh.

Phase 4: Instant Emergency Broadcast Override

1. Admin enters urgent text and clicks **Broadcast Alert** in the emergency bar.
2. Server issues a high-priority `EMERGENCY_OVERRIDE` packet across all displays.
3. Every active TV player immediately pauses background video, reveals an animated crimson alert banner, ramps volume to 80%, and plays alert audio.
4. Clicking **Clear Alert** emits `CLEAR_EMERGENCY`, restoring normal playback seamlessly.

Phase 5: Multi-Display Scaling & Lifecycle Decommissioning

- **Adaptive Grid Density:** Toggle between **Comfortable** (260px cards, 4 TVs/row) and **Compact Grid** (210px cards, 5-8 TVs/row) to view large display fleets without horizontal scrolling.
- **Individual Display Deletion:** Clicking the red **Delete TV** button on any card confirms decommissioning, signals `DEVICE_DELETED` over WebSocket, disconnects the TV, and removes it from storage.
- **One-Click Offline Pruning:** The **Clear Offline TVs** toolbar action purges disconnected or stale displays while keeping all streaming screens intact.

5. SECURITY ARCHITECTURE & VAPT VERIFICATION

The platform implements rigorous defense-in-depth protections aligned with OWASP Top 10 standards, verified through automated penetration testing:

OWASP RISK CATEGORY	IMPLEMENTED SECURITY COUNTERMEASURE	VERIFICATION & TEST RESULT
A01: Broken Access Control	`requireAuth` middleware guarding all mutation endpoints (broadcast, update, delete, upload).	Verified: Unauthenticated requests strictly receive HTTP 401 Unauthorized.
A07: Identification & Auth Failures	IP-based sliding rate limiter locking out brute-force attacks after 5 failed login attempts for 5 minutes.	Verified: Exceeding 5 attempts immediately returns HTTP 429 Too Many Requests.
A03: Injection & Stored XSS	Dual-layer defense: backend regex sanitization (strips `<>'"`) and frontend HTML entity encoding.	Verified: Script payloads rendered inert as sanitized string literals.
A04: Unrestricted File Upload	Multer extension whitelist (.mp4, .webm, .mkv) with 500MB cap and alphanumeric name sanitization.	Verified: Polyglot and executable uploads rejected with HTTP 400.
WebSocket DoS & Overload	`maxPayload: 64KB` restriction with integer index bounds validation on message handling.	Verified: Malformed or oversized frames terminated immediately.
Information Disclosure	Global Express error handler intercepts uncaught exceptions, suppressing internal stack traces.	Verified: Production errors return generic sanitized messages.

6. NETWORK TOPOLOGY & PORT ALLOCATION

To ensure reliable streaming across LAN, Enterprise Wi-Fi, and remote offices, configure the following ports:

PORT / PROTOCOL	SERVICE NAME	DIRECTION & SCOPE	OPERATIONAL FUNCTION
`TCP 8080`	HTTP Management	Inbound (LAN + VPN)	Admin console, player web app, media upload API, WebSocket control bus.
`TCP 8443`	HTTPS Secure API	Inbound (LAN + VPN)	Encrypted administration and browser screen-sharing capture.
`TCP 8889`	MediaMTX WHEP/WHIP	Inbound (LAN + VPN)	WebRTC signaling for sub-300ms live stream broadcasting and playback.
`UDP 8189`	WebRTC ICE Media	Bidirectional (LAN + VPN)	High-throughput video and audio packet transport for WebRTC streams.
`TCP 8888`	MediaMTX HLS	Inbound (LAN + VPN)	Low-latency HLS video delivery for legacy displays or restricted firewalls.
`TCP 8554`	MediaMTX RTSP	Inbound (Internal Network)	RTSP ingestion for IP security cameras, hardware decoders, and OBS.
`TCP 3001`	LG webOS SSAP	Outbound (to TV IP)	Automated TV subnet scanning, pairing prompt negotiation, and browser launch.

7. PRODUCTION DEPLOYMENT & INSTALLATION PACKAGES

Package 1: Docker Container Stack (Recommended)

Archive: `cisai-central-tv-docker.zip`

```
unzip cisai-central-tv-docker.zip
cd cisai-central-tv-docker
docker compose up -d
```

Pre-configured multi-container stack running Node.js 20 manager and MediaMTX with automatic restart on boot.

Package 2: Bare Metal / VM Systemd Services

Archive: `cisai-central-tv-baremetal-vm.zip`

```
unzip cisai-central-tv-baremetal-vm.zip
cd cisai-central-tv-baremetal-vm
sudo bash scripts/install-systemd.sh
```

Native Ubuntu/Debian deployment using systemd daemons and automated service management.

Package 3: Raspberry Pi Kiosk Auto-Boot Engine

Archive: `cisai-central-tv-raspberry-pi.zip`

```
unzip cisai-central-tv-raspberry-pi.zip
cd cisai-central-tv-raspberry-pi
sudo bash scripts/start-kiosk.sh http://<SERVER_IP>:8080/player.html
```

Configures Chromium kiosk mode, disables screen blanking, and activates hardware watchdog.

Default Administrative Credentials: Username: `admin` | Password: `admin123`
Change credentials in production via the `.env` file or environment variables.